

Investigación #1: Caso de Estudio

Diseño e Implementación del Sistema de Gestión Académica de la Universidad XUNA

Estudiantes:

Joanfer Hidalgo Chaves

Santiago Hernández Chaves

Justin Angulo Artavia

Escuela de Informática

Universidad Nacional de Costa Rica

EIF207, Estructuras de Datos

Docente:

Msc. Johnny Villalobos Murillo

10 de agosto de 2026

II Ciclo 2026

Índice

Introducción	5
Análisis del problema	6
Restricciones técnicas del proyecto	6
Preguntas arquitectónicas planteadas	7
Identificación de los datos presentes en el caso	9
Clasificación de los datos según su Tipo de Dato Abstracto	12
Diseño del registro Estudiante	15
IdentidadEstudiante	17
AcademicoEstudiante	17
ContactoEstudiante	17
Justificación de cada uno de los campos del registro	19
Diseño del arreglo que almacenará los estudiantes	21
Justificación de los números utilizados	21
Desglose exacto a partir de los structs definidos	21
Composición de una entrada del mapa de posiciones	23
Por qué cargar todo en memoria no es viable	23
Resumen (valores aproximados)	23
Representación gráfica del arreglo	25
Identificación de todas las funciones necesarias	27
Módulo archivo (persistencia)	27
Módulo de mapas de posiciones (RAM)	27
Módulo estudiante (lógica de negocio)	27

Módulo reportes (agregados y mantenimiento)	28
Descripción del propósito de cada función	29
Pseudocódigo de las funciones principales	31
registrarEstudiante(datos)	31
buscarEstudiantePorCarne(carne)	31
buscarPosicionPorCarne(carne)	32
buscarEstudiantesPorApellido(apellido)	32
eliminarEstudianteLogico(carne)	33
registrarPosicionEnMapa(carne, posicion)	33
fusionarRecienIngresados()	33
purgarInactivos()	34
Análisis del costo computacional de cada función	36
Módulo de mapas de posiciones	36
Módulo archivo (persistencia)	37
Módulo estudiante	37
Módulo reportes (agregados y mantenimiento)	38
Tabla resumen	39
Explicación de cómo se almacenará la información en un archivo secuencial	43
Propuesta de un archivo indexado que permita acelerar las búsquedas	44
Funcionamiento general del mapa de posiciones	44
Mapa por carné	45
Mapas por identificación y por apellido	45
Reorganización y persistencia	46
Diagramas	48

Mapa conceptual	49
Conclusiones	50

Introducción

En la actualidad, las organizaciones manejan grandes cantidades de información que deben mantenerse organizadas para facilitar su consulta, actualización, almacenamiento y recuperación. En instituciones como las universidades, esta necesidad adquiere especial importancia debido al volumen de datos personales y académicos que deben administrarse de forma constante.

Antes de desarrollar un sistema informático es necesario comprender el problema que se desea resolver, identificar los datos que serán administrados y definir estructuras adecuadas para representarlos. De igual forma, se debe considerar la eficiencia de las operaciones que se realizarán sobre dicha información, debido a que distintas soluciones pueden requerir diferentes cantidades de tiempo y recursos.

La presente investigación analiza el caso de la Universidad XUNA, una institución con aproximadamente 25 000 estudiantes que busca reemplazar el uso de formularios impresos, hojas electrónicas y programas independientes por un Sistema de Gestión Académica centralizado. El sistema deberá permitir administrar información personal y académica de los estudiantes, además de realizar operaciones como registrar, buscar, modificar, consultar y generar reportes.

A partir de este caso se estudiarán conceptos fundamentales relacionados con las estructuras de datos, entre ellos el dato, los Tipos de Datos Abstractos, los registros, los arreglos, las funciones, el costo computacional y los mecanismos de almacenamiento mediante archivos secuenciales e indexados. El objetivo es establecer una propuesta de diseño organizada y técnicamente justificada que sirva como base para el posterior desarrollo del sistema.

Análisis del problema

La Universidad XUNA cuenta con aproximadamente 25 000 estudiantes distribuidos entre distintas sedes regionales y más de cincuenta carreras. Durante varios años, la información estudiantil ha sido administrada mediante formularios impresos, hojas electrónicas y programas desarrollados de manera independiente por diferentes departamentos.

Esta forma de trabajo ha provocado que la información se encuentre distribuida en distintos medios y formatos. Como consecuencia, la universidad presenta problemas como expedientes duplicados, datos inconsistentes, errores en la información personal, dificultad para localizar estudiantes, retrasos durante los procesos de matrícula, lentitud para generar reportes y pérdida de información histórica.

Para solucionar esta situación se plantea el desarrollo de un Sistema de Gestión Académica que centralice la información de los estudiantes y permita administrarla de forma uniforme. El sistema deberá manejar tanto datos personales como académicos y permitir operaciones de registro, búsqueda, consulta, modificación, eliminación y generación de reportes.

El volumen de información también representa un aspecto importante del problema. Aunque actualmente la institución administra alrededor de 25 000 estudiantes, se espera que esta cantidad aumente con el paso del tiempo. Por esta razón, el diseño debe considerar no solamente la forma en que se representará cada estudiante, sino también cómo se almacenarán y localizarán grandes cantidades de registros sin realizar recorridos innecesarios sobre toda la información.

Además, los datos no pueden permanecer únicamente en la memoria principal, ya que desaparecerían al finalizar la ejecución del programa. Por lo tanto, el sistema también requiere un mecanismo de almacenamiento permanente que permita conservar la información entre ejecuciones y recuperarla cuando sea necesario.

Restricciones técnicas del proyecto

La solución propuesta debe ajustarse a una serie de restricciones establecidas en el caso de estudio. El sistema deberá desarrollarse en C++ utilizando programación estructurada y no podrá emplear Programación Orientada a Objetos.

Las entidades del sistema deberán representarse mediante registros definidos con `struct`, mientras que las operaciones deberán implementarse utilizando funciones independientes. De esta manera, no se permitirá utilizar clases, objetos, herencia, polimorfismo ni plantillas.

Para el almacenamiento de información en memoria solamente se podrán utilizar arreglos nativos del lenguaje. Por esta razón, quedan excluidas las estructuras proporcionadas por la Standard Template Library (STL), como `vector`, `list`, `map`, `set`, `queue` y otras estructuras similares.

El almacenamiento permanente deberá realizarse preferentemente mediante archivos binarios. Para manipular estos archivos se podrán utilizar funciones como `fopen()`, `fclose()`, `fread()`, `fwrite()`, `fseek()` y `ftell()`, permitiendo trabajar directamente con los registros almacenados en disco.

Finalmente, todas las funciones propuestas deberán incluir un análisis de su costo computacional mediante notación Big O. Esto obliga a considerar desde la etapa de diseño la eficiencia de cada operación y no únicamente su funcionamiento.

Preguntas arquitectónicas planteadas

A partir del problema y de las restricciones establecidas surgen varias decisiones que deben resolverse antes de implementar el sistema.

En primer lugar, es necesario determinar cómo representar los diferentes datos que pertenecen a un estudiante sin utilizar clases ni otras herramientas propias de la Programación Orientada a Objetos. Para ello, el diseño deberá organizar los datos mediante registros y definir con claridad el tipo correspondiente a cada campo.

También debe determinarse qué información resulta conveniente mantener en memoria principal. Cargar permanentemente todos los datos completos de los estudiantes puede aumentar considerablemente el consumo de memoria conforme crezca la matrícula, por lo que el diseño deberá buscar una forma de conservar en memoria únicamente la información necesaria para localizar los registros con eficiencia.

Otro problema consiste en las búsquedas. El sistema debe localizar estudiantes por

número de carné, número de identificación y apellido. Realizar un recorrido secuencial sobre todos los registros para cada consulta tendría un costo que aumenta directamente con la cantidad de estudiantes. Por esta razón, posteriormente será necesario definir estructuras auxiliares que permitan reducir el número de registros que deben examinarse durante una búsqueda.

Asimismo, la incorporación de nuevos estudiantes debe realizarse sin obligar al sistema a reorganizar por completo la información después de cada inserción. El diseño deberá equilibrar el costo de registrar nuevos estudiantes con la necesidad de mantener búsquedas eficientes.

Finalmente, se debe resolver la persistencia de la información. Los datos deberán almacenarse en archivos para evitar su pérdida al terminar el programa y, al mismo tiempo, deberá existir una estrategia que permita acceder directamente a los registros sin recorrer todo el archivo en cada consulta.

Estas decisiones constituyen la base de las estructuras de datos, registros, arreglos, funciones y mecanismos de almacenamiento que se desarrollan en las siguientes secciones del informe.

Identificación de los datos presentes en el caso

A partir de los requerimientos establecidos por la Oficina de Registro, se identifican catorce datos principales que deberán almacenarse para representar la información de cada estudiante dentro del Sistema de Gestión Académica.

Estos datos incluyen información de identificación personal, información de contacto y datos relacionados con la situación académica del estudiante. Cada uno cumple una función específica dentro de las operaciones que posteriormente realizará el sistema.

Dato	Descripción
Número de carné	Identificador asignado por la universidad a cada estudiante. Será uno de los principales datos utilizados para localizar y consultar un expediente.
Número de identificación	Documento de identificación personal del estudiante. El sistema también deberá permitir búsquedas utilizando este dato.
Nombre	Nombre o nombres del estudiante, necesarios para su identificación personal dentro del sistema.
Apellidos	Apellidos del estudiante. Además de formar parte de su identificación, el caso establece que deberán poder realizarse búsquedas por apellido.
Fecha de nacimiento	Fecha en la que nació el estudiante. Forma parte de la información personal que debe conservarse en su expediente.
Sexo	Dato utilizado para registrar el sexo correspondiente al estudiante dentro de su información personal.
Carrera	Carrera académica a la que pertenece el estudiante dentro de la universidad. Este dato permitirá, entre otras operaciones, mostrar estudiantes pertenecientes a una carrera específica.

Dato	Descripción
Nivel académico	Nivel académico en el que se encuentra actualmente el estudiante dentro de su proceso de formación.
Correo electrónico	Dirección de correo utilizada como medio de contacto con el estudiante.
Número telefónico	Número utilizado para contactar al estudiante por vía telefónica.
Dirección	Información correspondiente al lugar de residencia o dirección registrada del estudiante.
Cantidad de créditos aprobados	Cantidad total de créditos que el estudiante ha aprobado. Este dato puede actualizarse conforme avance en su carrera y también utilizarse para realizar comparaciones entre estudiantes.
Promedio general	Promedio académico acumulado del estudiante. Debe poder actualizarse y utilizarse tanto para consultas individuales como para cálculos y reportes institucionales.
Estado del estudiante	Indica la condición actual del estudiante dentro de la universidad y permite diferenciar, entre otros casos, estudiantes activos e inactivos.

Además de almacenarse como parte del expediente estudiantil, algunos de estos datos tienen una importancia particular debido a las operaciones solicitadas en el caso de estudio. El número de carné y el número de identificación deberán permitir localizar estudiantes de manera individual, mientras que los apellidos permitirán realizar búsquedas que pueden devolver varios resultados.

Por otra parte, datos como la carrera, los créditos aprobados, el promedio general y el estado del estudiante serán utilizados en consultas, actualizaciones y generación de información

académica. Esta identificación de los datos constituye la base para determinar, en la siguiente sección, la forma lógica en que cada uno será representado dentro del sistema.

Clasificación de los datos según su Tipo de Dato Abstracto

Una vez identificados los datos que forman parte del expediente de cada estudiante, es necesario determinar la naturaleza de la información que representa cada uno. Esta clasificación permite definir posteriormente la forma en que los datos serán almacenados y manipulados dentro del programa.

Para este diseño se consideran tipos como entero, real, cadena de caracteres, dato enumerado y fecha. La clasificación lógica de cada dato se presenta en la siguiente tabla.

Dato	Tipo de dato	Justificación
Número de carné	Entero	Representa el identificador numérico asignado por la universidad a cada estudiante. Se utilizará principalmente como llave de búsqueda y referencia del registro.
Número de identificación	Cadena de caracteres	Aunque está compuesto principalmente por dígitos, funciona como un identificador y puede variar en longitud o incluir caracteres según el tipo de documento. Por esta razón se trata como información textual.
Nombre	Cadena de caracteres	Está compuesto por una secuencia de caracteres alfabéticos y forma parte de la información personal del estudiante.
Apellidos	Cadena de caracteres	Corresponde a información textual. En el diseño posterior podrá separarse en diferentes campos para facilitar búsquedas y ordenamientos.

Dato	Tipo de dato	Justificación
Fecha de nacimiento	Fecha	Está formada por varios valores relacionados entre sí, específicamente día, mes y año. Por esta razón resulta conveniente representarla mediante un tipo compuesto destinado al manejo de fechas.
Sexo	Enumerado	Corresponde a un conjunto limitado de valores posibles, por lo que puede representarse mediante un dominio cerrado en lugar de utilizar texto libre.
Carrera	Entero de referencia	Puede almacenarse mediante un código numérico que identifique una carrera dentro de un catálogo, evitando repetir el nombre completo de la carrera en cada registro.
Nivel académico	Enumerado	Representa una categoría dentro de un conjunto limitado de niveles académicos definidos por el sistema.
Correo electrónico	Cadena de caracteres	Puede contener letras, números y símbolos, por lo que debe almacenarse como una secuencia de caracteres.
Número telefónico	Cadena de caracteres	Aunque contiene dígitos, no representa una cantidad sobre la cual deban realizarse operaciones matemáticas. Se utiliza únicamente como dato de contacto.

Dato	Tipo de dato	Justificación
Dirección	Cadena de caracteres	Contiene texto de longitud variable que puede incluir letras, números, espacios y otros caracteres.
Cantidad de créditos aprobados	Entero	Representa una cantidad discreta de créditos acumulados por el estudiante y puede utilizarse en operaciones de comparación y actualización.
Promedio general	Real	Puede contener una parte decimal, por lo que requiere un tipo numérico capaz de representar valores fraccionarios.
Estado del estudiante	Enumerado	Representa un conjunto limitado de condiciones posibles del estudiante, como activo o inactivo.

Esta clasificación distingue entre los datos que representan cantidades numéricas y aquellos que, aunque puedan contener dígitos, funcionan únicamente como identificadores. Por ejemplo, el número telefónico y el número de identificación no se utilizan para realizar cálculos, mientras que la cantidad de créditos aprobados y el promedio general sí forman parte de operaciones académicas.

También se identifican datos cuyo dominio está limitado a un conjunto determinado de valores, como el sexo, el nivel académico y el estado del estudiante. Estos pueden tratarse como tipos enumerados para evitar el almacenamiento de valores arbitrarios.

La clasificación anterior establece la representación lógica de los datos. En las siguientes secciones estos tipos se traducen a estructuras concretas del lenguaje C++, respetando las restricciones del proyecto y las necesidades de almacenamiento del sistema.

Diseño del registro Estudiante

El enunciado de la investigación pide administrar catorce datos por estudiante:

- número de carné
- identificación
- nombre
- apellidos
- fecha de nacimiento
- sexo
- carrera
- nivel académico
- correo electrónico
- número telefónico
- dirección
- cantidad de créditos aprobados
- promedio general
- estado del estudiante.

Nota: en el diseño final este último campo se separó en dos, estadoAcademico y marcadoParaPurga (sección), ya que ambas cosas no son lo mismo: el estado académico de un estudiante (activo, en pausa, con permiso) no implica que su expediente deba eliminarse.

El enunciado no propone ninguna forma de organizar estos datos, solo se pide que sean guardados.

La figura 1 muestra el desglose exacto de cada struct: los campos en el orden en que quedan declarados, su tamaño y el offset donde inicia cada uno dentro del registro, incluyendo los bytes de *padding* que agrega el compilador para mantener alineados los campos numéricos (detalle explicado en la sección).

La primer idea del equipo fue juntar los catorce campos en un único registro. Al revisarlo con cuidado apareció un problema:

Esos catorce campos no cambian con la misma frecuencia ni se usan juntos en las mismas operaciones

Por ejemplo el nombre y la fecha de nacimiento de un estudiante casi nunca cambian una vez matriculado. El promedio y los créditos aprobados se actualizan varias veces durante la carrera del estudiante, cada período lectivo. El correo, el teléfono y la dirección se consultan juntos normalmente solo cuando hace falta contactar al estudiante, y casi nunca al mismo tiempo que se actualiza el promedio.

Juntar estos tres en un solo registro haría que actualizar el promedio de un estudiante, algo que ocurre constantemente, obliga también a leer y reescribir su nombre, su fecha de nacimiento, etc; aunque ninguno de esos datos haya cambiado.

Por esa razón se decidió dividir el registro en tres structs independientes, cada uno guardado en su propio archivo binario (*identidad.dat*, *academico.dat*, *contacto.dat*). El criterio de separación es la frecuencia y el patrón de uso de cada grupo de campos. Nos basamos en como al crear bases de datos relacionales se separan los datos distintas tablas físicas. Los tres structs se mantienen alineados por posición:

1. El registro en la posición i de *identidad.dat*
2. El registro en la posición i de *academico.dat*
3. El registro en la posición i de *contacto.dat*

Es decir, el *indice* en cada archivo corresponde al mismo estudiante evitando generar 3 indices y búsquedas distintas.

IdentidadEstudiante

Datos que identifican al estudiante como persona

```

1 typedef struct {
2     int     carne;
3     char    identificacion[13];
4     char    nombre[21];
5     char    apellidos[41];
6     Fecha   fechaNacimiento;
7     int     sexo;
8 }
9 IdentidadEstudiante;
```

AcademicoEstudiante

Datos de la situación académica actual del estudiante, que cambian cada período lectivo.

```

1 typedef struct {
2     int     carne;
3     int     carrera;
4     int     nivelAcademico;
5     int     creditosAprobados;
6     float   promedio;
7     int     estadoAcademico;
8     int     marcadoParaPurga;
9 }
10 AcademicoEstudiante;
```

ContactoEstudiante

Datos usados para comunicarse con el estudiante, que cambian con poca frecuencia.

```
1 typedef struct {  
2     int    carne;  
3     char   correo[41];  
4     char   telefono[9];  
5     char   direccion[81];  
6 }  
7 ContactoEstudiante;
```

Justificación de cada uno de los campos del registro

Todos los campos de tipo texto se implementan como `char []` de tamaño fijo ya que esto permite calcular el `offset` de cualquier registro dentro de su archivo mediante

$$offset = posicin \times sizeof(struct)$$

Cada arreglo de caracteres reserva un byte adicional para el carácter nulo (`'\0'`), que en C marca el final de la cadena:

Campo	Tipo	Tamaño	Justificación
carne	int	4 B	Llave primaria, requerida explícitamente por el caso de estudio
identificacion	char[13]	13 B	Cubre el caso más ancho entre cédula física (9 dígitos) y pasaporte alfanumérico: 12 caracteres.
nombre	char[21]	21 B	20 caracteres útiles, margen razonable para nombres compuestos.
apellidos	char[41]	41 B	Un solo campo, como lo pide el enunciado: 40 caracteres útiles cubren primer y segundo apellido juntos.
fechaNacimiento	struct Fecha	12 B	TAD propio (día, mes, año como int), reutilizable y con sus propias reglas de validez.
sexo	enum (int)	4 B	Dominio cerrado paraa evitar inconsistencias de tamanos ->representado como entero en vez de texto libre.

Campo	Tipo	Tamaño	Justificación
carrera	int	4 B	Referencia a un catálogo de carreras en vez de repetir el nombre completo en cada registro.
nivelAcademico	enum (int)	4 B	Dominio cerrado (pregrado, grado, posgrado, etc).
creditosAprobados	int	4 B	Cantidad entera, sin componente fraccionario.
promedio	float	4 B	Requiere parte decimal.
estadoAcademico	enum (int)	4 B	Condición académica del estudiante (activo, inactivo, permiso, etc); no indica si el registro debe eliminarse
marcadoParaPurga	int (booleano)	4 B	0 o 1; bandera de borrado lógico independiente del estado académico, para que purgarInactivos() nunca borre un expediente solo por estar en pausa o con permiso
correo	char[41]	41 B	Margen suficiente para direcciones institucionales y personales comunes.
telefono	char[9]	9 B	8 dígitos (formato costarricense) más el terminador nulo.
direccion	char[81]	81 B	Texto libre de longitud variable, con un tope generoso para evitar problemas

Diseño del arreglo que almacenará los estudiantes

El arreglo de registros completos no se carga en memoria principal: está en disco, en los tres archivos binarios descritos en la sección anterior. Lo que sí reside en RAM en todo momento es un arreglo, al que llamamos **mapa de posiciones**: guarda únicamente la llave de búsqueda, la posición física del registro en disco y su estado. El resto de los datos del estudiante deben extraerse del archivo.

Justificación de los números utilizados

Los tamaños que se usan en las estimaciones a continuación provienen directamente de los campos y tamaños definidos en la sección . A continuación el razonamiento de cada valor:

1. Se revisan los tres *structs* que componen el registro: `IdentidadEstudiante`, `AcademicoEstudiante` y `ContactoEstudiante`
2. Para cada campo se toma el tamaño indicado en la justificación (por ejemplo: `int carne = 4 B`)
3. Sumando los tamaños de los campos de cada *struct* se obtiene un tamaño por *struct*; sumando los tres *structs* se obtiene el tamaño total por estudiante almacenado en los archivos.
4. Para las entradas del mapa de posiciones se estima el espacio requerido por: la llave (carné), la posición dentro del archivo y un indicador de estado. Los valores utilizados para la estimación (4 B, 8 B y 4 B) son redondeos razonables.

Desglose exacto a partir de los structs definidos

Usando los tamaños de campo especificados en la sección se obtiene el siguiente desglose:

■ **IdentidadEstudiante:**

- `int carne = 4 B` (se elige 4 B ya que 1 B = 8 bits, y 4 B permiten representar enteros de hasta 2,147,483,647, suficiente para los carnés de estudiantes)
- `char identificacion[13] = 13 B`
- `char nombre[21] = 21 B`
- `char apellidos[41] = 41 B`
- `Fecha (3 ints) = 12 B`
- `int sexo = 4 B`

Suma cruda de campos = $4 + 13 + 21 + 41 + 12 + 4 = 95$ B. Sin embargo, el compilador no empaqueta los `char []` pegados a los `int/Fecha`: en la mayoría de plataformas de 32/64 bits los campos de 4 B se alinean a múltiplos de 4, lo que agrega 1 B de *padding* después de `identificacion[13]` y después de `apellidos[41]` (13 y 41 no son múltiplos de 4). El `sizeof(IdentidadEstudiante)` real del compilador es **96 B**, no 95 B.

- **AcademicoEstudiante**: siete campos numéricos de 4 B cada uno, sumando 28 B. Total académico = $4 \times 7 = 28$ B (justificación en la sección)

- **ContactoEstudiante**:

- `int carne = 4 B`
- `char correo[41] = 41 B`
- `char telefono[9] = 9 B`
- `char direccion[81] = 81 B`

Suma cruda = $4 + 41 + 9 + 81 = 135$ B. Por la misma razón de alineación, `sizeof(ContactoEstudiante)` real es **136 B**.

Suma con padding real (registro completo por estudiante en los tres archivos): $96 + 28 + 136 = 260$ B.

Composición de una entrada del mapa de posiciones

Cada entrada del mapa contiene tres valores principales: la llave de búsqueda (el carné), la posición del registro dentro del archivo (cuántos bytes desde el inicio del archivo) y un indicador de estado.

Al hacer estimaciones se utilizan valores redondeados para facilitar el cálculo:

- Llave (carné): 4 B
- Posición en el archivo: 8 B (1 B = 8 bits, 8 B permiten representar posiciones de hasta 18,446,744,073,709,551,615 bytes, suficiente para archivos muy grandes)
- Estado: 4 B (pequeño indicador/flag, 4 B es suficiente)

Sumando esos valores, cada entrada ocupa aproximadamente 16 B. Manteniendo tres mapas distintos (por carné, por identificación y por apellido), el coste en memoria por estudiante es:

$$16 \text{ B por entrada} \times 3 = 48 \text{ B}$$

Estas cifras son estimaciones orientativas usadas para comparar el tamaño relativo entre almacenar los registros completos en disco y mantener solo los mapas en RAM.

Por qué cargar todo en memoria no es viable

Almacenar los registros completos directamente en RAM escala de manera ineficiente a medida que crece el volumen de datos (n), consumiendo recursos esenciales del sistema. Por ello, reservar la memoria principal únicamente para el índice (mapa de posiciones) y consultar los detalles en disco por demanda permite optimizar drásticamente el espacio utilizado.

Resumen (valores aproximados)

Los valores de la tabla siguiente se obtienen aplicando las fórmulas:

- Registro completo: $260 \text{ B} \times n$
- Mapas en RAM: $48 \text{ B} \times n$

Tabla 4

Comparación del consumo de almacenamiento según la cantidad de estudiantes.

Cantidad (n)	Registro completo (Disco)		Mapas de posiciones (RAM)	
	Bytes	MiB (≈)	Bytes	MiB (≈)
25 000	6 500 000	6.20	1 200 000	1.14
100 000	26 000 000	24.80	4 800 000	4.58
1 000 000	260 000 000	247.96	48 000 000	45.78

Se muestran tanto los bytes totales como la conversión a MiB ($1 \text{ MiB} = 2^{20} \text{ bytes}$).

Interpretación: Con las hipótesis indicadas, el contenido completo en disco ocupa aproximadamente 5 veces más espacio que los tres mapas en memoria. Aunque las cifras son estimaciones, ilustran que mantener solo los mapas en RAM reduce significativamente el uso de memoria sin perder la capacidad de localizar y recuperar registros desde disco.

Representación gráfica del arreglo

La arquitectura de este arreglo se explica en las secciones y ; para evitar redundancia en esta seccion no se explicara la justificacion aun.

La figura 2 muestra las dos zonas del mapa por carné descritas en la subsección , cada una como un arreglo separado:

1. La **zona ordenada** (índices 0 a 3), ordenada por carné.
2. La **lista de recién ingresados** (índices 0 y 1, con los carnés 1015 y 1003 que todavía no se integraron a la zona ordenada).

Cada celda, en cualquiera de las dos zonas, tiene la misma composición de tres campos justificada en la subsección . La celda en rojo (carné 1004) es el mismo ejemplo de estado inactivo usado en la sección . La flecha punteada entre ambas zonas representa lo que ocurre cuando una búsqueda no encuentra la llave en la zona ordenada: también se revisa la lista de recién ingresados antes de reportar que el estudiante no existe.

La figura 3 agrega los tres archivos en disco (sección) a la misma estructura de la figura anterior. Las flechas punteadas son el cálculo de offset de la sección aplicado a cada entrada del mapa:

- Los carnés 1001 y 1002 (índices 0 y 1 de la zona ordenada) apuntan directamente a las posiciones 0 y 1 en disco.
- El carné 1007 (índice 3 de la zona ordenada) en cambio apunta a la posición 2, y el carné 1004 (inactivo) a la posición 3: el índice dentro del mapa no siempre coincide con la posición física en disco, que es justo la razón por la que cada entrada guarda su propio offset en vez de asumir que ambos números son el mismo.
- El carné 1015 (lista de recién ingresados) apunta a la posición 4, la más reciente.

Las tres columnas de cada posición (`identidad.dat`, `academico.dat`, `contacto.dat`) ilustran que comparten índice sin necesitar una llave adicional entre ellas, también justificado en la sección .

Identificación de todas las funciones necesarias

Módulo archivo (persistencia)

Función	Entrada → Salida
abrirArchivo(nombre,modo)	nombre, modo → FILE*
cerrarArchivo(fp)	FILE* → void
calcularOffset(posicion,tamRegistro)	posición, tamaño → offset en bytes
leerRegistroEnOffset(fp,offset,destino)	offset → registro leído
escribirRegistroEnOffset(fp,offset,registro)	offset, registro → éxito o error
agregarRegistroAlFinal(fp,registro)	registro → offset asignado
contarRegistros(fp,tamRegistro)	sin entrada → cantidad total

Módulo de mapas de posiciones (RAM)

Función	Entrada → Salida
cargarMapaDePosiciones()	sin entrada → mapa cargado desde disco
buscarPosicionPorCarne(carne)	carne → posición o "no encontrado"
registrarPosicionEnMapa(carne,posicion)	carne, posición → éxito
fusionarRecienIngresados()	sin entrada → mapa con zona ordenada y lista de recién ingresados fusionadas
marcarInactivoEnMapa(carne)	carne → éxito
construirMapaPorCampo(campo)	campo → mapa ordenado por ese campo
buscarPosicionesPorCampo(campo,valor)	valor → lista de posiciones
guardarMapasEnDisco()	sin entrada → mapas guardados en .idx

Módulo estudiante (lógica de negocio)

Función	Entrada → Salida
registrarEstudiante(datos)	datos completos → éxito
buscarEstudiantePorCarne(carne)	carne → estudiante
buscarEstudiantePorIdentificacion(id)	identificación → estudiante
buscarEstudiantesPorApellido(apellido)	apellido → lista de estudiantes
modificarEstudiante(carne,nuevosDatos)	carne, datos → éxito
eliminarEstudianteLogico(carne)	carne → éxito
mostrarTodosLosEstudiantes()	sin entrada → recorrido completo
mostrarEstudiantesPorCarrera(carrera)	carrera → lista
actualizarPromedio(carne,promedio)	carne, valor → éxito
actualizarCreditosAprobados(carne,credit os)	carne, valor → éxito
consultarExpedienteCompleto(carne)	carne → identidad más académico más con- tacto

Módulo reportes (agregados y mantenimiento)

Función	Entrada → Salida
calcularPromedioInstitucional()	sin entrada → promedio
encontrarMayorPromedio()	sin entrada → estudiante
encontrarMayorCreditos()	sin entrada → estudiante
generarReporteEstadistico()	sin entrada → resumen
purgarInactivos()	sin entrada → archivo reconstruido sin reg- istros con estado en cero

Descripción del propósito de cada función

Función	Propósito
registrarEstudiante	Agrega un nuevo estudiante: escribe su registro en los tres archivos y lo incorpora al mapa de posiciones.
buscarEstudiantePorCarne	Recupera el expediente (identidad, académico y contacto) a partir del carné.
buscarEstudiantePorIdentificación	Igual que la anterior, pero usando el número de identificación como llave.
buscarEstudiantesPorApellido	Recupera todos los estudiantes que comparten un apellido dado.
modificarEstudiante	Sobrescribe los campos de un estudiante existente, sin cambiar su posición ni su carné.
eliminarEstudianteLogico	Marca a un estudiante como inactivo sin borrar físicamente su registro.
mostrarTodosLosEstudiantes	Recorre y despliega el listado completo de estudiantes activos.
mostrarEstudiantesPorCarrera	Filtra y despliega los estudiantes de una carrera específica.
actualizarPromedio	Recalcula y guarda el promedio general de un estudiante.
actualizarCreditosAprobados	Recalcula y guarda la cantidad de créditos aprobados de un estudiante.
consultarExpedienteCompleto	Reúne los tres registros (identidad, académico, contacto) de un mismo estudiante.
calcularPromedioInstitucional	Calcula el promedio general de toda la población estudiantil activa.

Función	Propósito
encontrarMayorPromedio	Ubica al estudiante con el promedio más alto.
encontrarMayorCreditos	Ubica al estudiante con más créditos aprobados.
generarReporteEstadistico	Produce un resumen agregado (totales por carrera, nivel, etc.).
purgarInactivos	Reconstruye el archivo físico eliminando de forma definitiva los registros marcados como inactivos.
cargarMapaDePosiciones	Carga o reconstruye el mapa de carné en memoria al iniciar el sistema.
buscarPosicionPorCarne	Ubica, dentro del mapa, la posición de un registro a partir de su carné.
registrarPosicionEnMapa	Agrega una nueva entrada al mapa sin reordenar la zona ordenada.
fusionarRecienIngresados	Junta la lista de recién ingresados con la zona ordenada y la deja ordenada de nuevo.
construirMapaPorCampo	Genera un mapa ordenado por un campo distinto al carné (identificación o apellido).
buscarPosicionesPorCampo	Ubica una o varias posiciones a partir de un valor de un campo secundario.

Pseudocódigo de las funciones principales

El pseudocódigo se escribe en sintaxis C simplificada: condicionales, ciclos y llamadas a función explícitas.

registrarEstudiante(datos)

```

1  funcion registrarEstudiante(datos):
2      si no validarFormato(datos):
3          retornar ERROR
4
5      posicion = buscarEspacioLibre()
6
7      si posicion == NO_HAY_ESPACIO:
8          posicion = expandirVector()
9
10     escribirRegistroEnOffset(identidad.dat, posicion, datos.identidad)
11     escribirRegistroEnOffset(academico.dat, posicion, datos.academico)
12     escribirRegistroEnOffset(contacto.dat, posicion, datos.contacto)
13
14     registrarPosicionEnMapa(datos.carne, posicion)
15
16     retornar EXITO

```

buscarEstudiantePorCarne(carne)

```

1  funcion buscarEstudiantePorCarne(carne):
2      posicion = buscarPosicionPorCarne(carne)
3
4      si posicion == NO_ENCONTRADO:
5          retornar NO_ENCONTRADO
6

```

```

7  identidad = leerRegistroEnOffset(identidad.dat, posicion)
8  academico = leerRegistroEnOffset(academico.dat, posicion)
9  contacto  = leerRegistroEnOffset(contacto.dat, posicion)
10
11  retornar ensamblarEstudiante(identidad, academico, contacto)

```

buscarPosicionPorCarne(carne)

```

1  funcion buscarPosicionPorCarne(carne):
2      posicion = busquedaBinaria(zonaOrdenada, carne)
3
4      si posicion != NO_ENCONTRADO:
5          retornar posicion
6
7      para i desde 0 hasta longitud(listaRecienIngresados) - 1:
8          si listaRecienIngresados[i].carne == carne:
9              retornar listaRecienIngresados[i].posicion
10
11  retornar NO_ENCONTRADO

```

buscarEstudiantesPorApellido(apellido)

```

1  funcion buscarEstudiantesPorApellido(apellido):
2      si mapaApellido.desactualizado:
3          mapaApellido = construirMapaPorCampo(APELLIDO)
4
5      posiciones = buscarPosicionesPorCampo(mapaApellido, apellido)
6
7      resultados = listaVacía()
8
9      para cada posicion en posiciones:

```

```

10     identidad = leerRegistroEnOffset(identidad.dat, posicion)
11     agregar(resultados, identidad)
12
13     retornar resultados

```

eliminarEstudianteLogico(carne)

```

1  funcion eliminarEstudianteLogico(carne):
2      posicion = buscarPosicionPorCarne(carne)
3
4      si posicion == NO_ENCONTRADO:
5          retornar ERROR
6
7      marcarInactivoEnMapa(carne)
8
9      academico = leerRegistroEnOffset(academico.dat, posicion)
10     academico.marcadoParaPurga = 1
11     escribirRegistroEnOffset(academico.dat, posicion, academico)
12
13     retornar EXITO

```

registrarPosicionEnMapa(carne, posicion)

```

1  funcion registrarPosicionEnMapa(carne, posicion):
2      agregarAlFinal(listaRecienIngresados, carne, posicion)
3
4      si longitud(listaRecienIngresados) >= 0.10 * longitud(zonaOrdenada):
5          fusionarRecienIngresados()

```

fusionarRecienIngresados()

```

1  funcion fusionarRecienIngresados():

```

```

2   zonaOrdenada = mezclarOrdenado(zonaOrdenada, listaRecienIngresados)
3   listaRecienIngresados = vacio()

```

purgarInactivos()

```

1  funcion purgarInactivos():
2      crear identidadTemp.dat, academicoTemp.dat, contactoTemp.dat
3
4      para cada posicion desde 0 hasta contarRegistros() - 1:
5          academico = leerRegistroEnOffset(academico.dat, posicion)
6
7          si academico.marcadoParaPurga != 1:
8              identidad = leerRegistroEnOffset(identidad.dat, posicion)
9              contacto  = leerRegistroEnOffset(contacto.dat, posicion)
10             agregarRegistroAlFinal(identidadTemp.dat, identidad)
11             agregarRegistroAlFinal(academicoTemp.dat, academico)
12             agregarRegistroAlFinal(contactoTemp.dat, contacto)
13
14             // si marcadoParaPurga es 1, el registro se descarta y no pasa al archivo
15             nuevo
16
17             // (independiente de estadoAcademico: un estudiante en pausa o con
18             permiso
19
20             // no se marca para purga, así que nunca se elimina solo por su estado
21             academico)
22
23             reemplazarArchivo(identidad.dat, identidadTemp.dat)
24             reemplazarArchivo(academico.dat, academicoTemp.dat)
25             reemplazarArchivo(contacto.dat, contactoTemp.dat)
26
27             // las posiciones cambiaron: reconstruir los tres mapas desde cero,
28             // no se puede reutilizar la lista de recién ingresados anterior

```

```
23  mapaCarne = reconstruirMapa(identidad.dat, CARNE)
24  mapaIdentificacion = reconstruirMapa(identidad.dat, IDENTIFICACION)
25  mapaApellido = reconstruirMapa(identidad.dat, APELLIDO)
26
27  retornar EXITO
```

Análisis del costo computacional de cada función

El análisis siguiente reporta dos cosas para cada función:

1. la clase asintótica (notación Big O)
2. el costo temporal real que esa clase esconde

Esta distinción importa porque una función puede clasificarse como $O(n)$ y aun así estar ejecutando, por ejemplo, $n + \log n$ o $3n$ operaciones reales. La notación Big O descarta constantes y términos de menor orden, pero esas operaciones se siguen ejecutando en la práctica.

Módulo de mapas de posiciones

Definiendo n como la cantidad de registros ordenados y k como la cantidad de registros recién ingresados

Función	Algoritmo	Costo real	Big O
buscarPosicionPorCarne	Binaria en zona ordenada, lineal en lista de recién ingresados si no hay coincidencia	$\log n + k$	$O(\log n) + O(k)$
registrarPosicionEnMapa	Inserción al final de la lista de recién ingresados; dispara fusionarRecienIngresados cada $0,1n$ inserciones	1 operación (amortizado)	$O(\log n)$ amortizado
fusionarRecienIngresados	Ordenar solo los k recién ingresados y mezclar linealmente con la zona ordenada (n), que ya está ordenada	$k \log k + n$	$O(n + k \log k)$
buscarPosicionesPorCam po	Binaria en zona ordenada, lineal en lista de recién ingresados	$\log n + k$	$O(\log n) + O(k)$

Función	Algoritmo	Costo real	Big O
construirMapaPorCampo (apellido)	Copia de n llaves más ordenamiento	$n + n \log n$	$O(n \log n)$

Módulo archivo (persistencia)

Función	Algoritmo	Costo real	Big O
leerRegistroEnOffset	fseek más fread de un registro de tamaño fijo	1 operación de E/S	$O(1)$
escribirRegistroEnOffset	fseek más fwrite de un registro de tamaño fijo	1 operación de E/S	$O(1)$
purgarInactivos	Recorrido secuencial de n registros, reescritura de los activos ($a \leq n$) y reconstrucción de 3 mapas	$n + a + n \log n$	$O(n \log n)$

Módulo estudiante

Función	Algoritmo	Costo real	Big O
registrarEstudiante	Inserción en el mapa más escritura de 3 registros de tamaño fijo	$1 + 3$	$O(\log n)$ amortizado
buscarEstudiantePorCarn e	Búsqueda en el mapa más 1 lectura en offset	$\log n + 1$	$O(\log n)$
buscarEstudiantesPorApe ellido	Búsqueda en el mapa de apellido más k lecturas de coincidencias	$\log n + k$	$O(\log n)$ $+O(k)$
modificarEstudiante	Búsqueda en el mapa más escritura en offset existente	$\log n + 1$	$O(\log n)$

Función	Algoritmo	Costo real	Big O
eliminarEstudianteLogico	Búsqueda en el mapa más marcar marcadoParaPurga = 1 en mapa y archivo	$\log n + 2$	$O(\log n)$
consultarExpedienteCompleto	Búsqueda en el mapa más 3 lecturas (identidad, académico, contacto)	$\log n + 3$	$O(\log n)$
mostrarTodosLosEstudiantes	Recorrido secuencial completo	n	$O(n)$
mostrarEstudiantesPorCarrera	Recorrido secuencial con filtro	n comparaciones	$O(n)$
actualizarPromedio/actualizarCreditosAprobados	Búsqueda en el mapa más escritura en academico.dat	$\log n + 1$	$O(\log n)$

Se decidió que para mostrarEstudiantesPorCarrera no se usaría un mapa aparte pero si se tuviera un archivo de índices por carrera se podría reducir el costo de $O(n)$ a $O(r)$ donde r es la cantidad de estudiantes de esa carrera.

Módulo reportes (agregados y mantenimiento)

Función	Algoritmo	Costo real	Big O
calcularPromedioInstitucional	Recorrido secuencial con contador	n sumas más 1 división	$O(n)$
encontrarMayorPromedio	Recorrido secuencial con máximo	n comparaciones	$O(n)$
encontrarMayorCreditos	Recorrido secuencial con máximo	n comparaciones	$O(n)$

Función	Algoritmo	Costo real	Big O
generarReporteEstadistico o	Recorrido secuencial con múltiples contadores (por carrera, por nivel, etc)	$n \times c$, c = cantidad de categorías	$O(n)$

El costo real de generarReporteEstadistico es $n \times c$ y no simplemente n : por cada uno de los n registros se actualizan c contadores. Esto hace que el costo real sea mayor que un recorrido con un solo contador pero ahorra tiempo de ejecucion.

Tabla resumen

La siguiente tabla consolida en un solo lugar todas las funciones propuestas de los cuatro módulos anteriores junto con su costo computacional, para facilitar su consulta como referencia rápida.

Tabla 14

Resumen de funciones propuestas y su costo computacional

Función	Algoritmo	Costo real	Big O
<i>Módulo archivo (persistencia)</i>			
abrirArchivo(nombre,modo)	Llamada directa a <code>fopen</code>	1 operación de E/S	$O(1)$
cerrarArchivo(fp)	Llamada directa a <code>fclose</code>	1 operación de E/S	$O(1)$
calcularOffset(posicion,tamRegistro)	Multiplicación directa	1 operación	$O(1)$
leerRegistroEnOffset(fp,offset,destino)	<code>fseek</code> más <code>fread</code> de un registro de tamaño fijo	1 operación de E/S	$O(1)$
escribirRegistroEnOffset(fp,offset,regi stro)	<code>fseek</code> más <code>fwrite</code> de un registro de tamaño fijo	1 operación de E/S	$O(1)$

Función	Algoritmo	Costo real	Big O
agregarRegistroAlFinal(fp,registro)	fseek al final más fwrite	1 operación de E/S	$O(1)$
contarRegistros(fp,tamRegistro)	ftell sobre el tamaño del archivo dividido entre el tamaño del struct	1 operación	$O(1)$
purgarInactivos()	Recorrido secuencial de n registros, reescritura de los activos ($a \leq n$) y reconstrucción de 3 mapas	$n + a + n \log n$	$O(n \log n)$
<i>Módulo de mapas de posiciones</i>			
cargarMapaDePosiciones()	Lectura secuencial del archivo .idx o reconstrucción desde los archivos de datos	n	$O(n)$
buscarPosicionPorCarne(carne)	Binaria en zona ordenada, lineal en lista de recién ingresados si no hay coincidencia	$\log n + k$	$O(\log n) + O(k)$
registrarPosicionEnMapa(carne,posicion)	Inserción al final de la lista de recién ingresados; dispara la fusión cada $0,1n$ inserciones	1 operación (amortizado)	$O(\log n)$ amortizado
fusionarRecienIngresados()	Ordenar solo los k recién ingresados y mezclar linealmente con la zona ordenada (n), ya ordenada	$k \log k + n$	$O(n + k \log k)$
marcarInactivoEnMapa(carne)	Búsqueda en el mapa más escritura de un campo	$\log n + 1$	$O(\log n)$
construirMapaPorCampo(campo)	Copia de n llaves más ordenamiento	$n + n \log n$	$O(n \log n)$

Función	Algoritmo	Costo real	Big O
buscarPosicionesPorCampo(campo,valor)	Binaria en zona ordenada, lineal en lista de recién ingresados	$\log n + k$	$O(\log n) + O(k)$
guardarMapasEnDisco()	Escritura secuencial de las n entradas de cada mapa	n	$O(n)$
<i>Módulo estudiante</i>			
registrarEstudiante(datos)	Inserción en el mapa más escritura de 3 registros de tamaño fijo	$1 + 3$	$O(\log n)$ amortizado
buscarEstudiantePorCarne(carne)	Búsqueda en el mapa más 1 lectura en offset	$\log n + 1$	$O(\log n)$
buscarEstudiantePorIdentificacion(id)	Búsqueda en el mapa de identificación más 1 lectura en offset	$\log n + 1$	$O(\log n)$
buscarEstudiantesPorApellido(apellido)	Búsqueda en el mapa de apellido más k lecturas de coincidencias	$\log n + k$	$O(\log n) + O(k)$
modificarEstudiante(carne,nuevosDatos)	Búsqueda en el mapa más escritura en offset existente	$\log n + 1$	$O(\log n)$
eliminarEstudianteLogico(carne)	Búsqueda en el mapa más marcar marcadoParaPurga = 1 en mapa y archivo	$\log n + 2$	$O(\log n)$
mostrarTodosLosEstudiantes()	Recorrido secuencial completo	n	$O(n)$
mostrarEstudiantesPorCarrera(carrera)	Recorrido secuencial con filtro	n comparaciones	$O(n)$
actualizarPromedio(carne,promedio)	Búsqueda en el mapa más escritura en academico.dat	$\log n + 1$	$O(\log n)$

Función	Algoritmo	Costo real	Big O
actualizarCreditosAprobados(carne, creditos)	Búsqueda en el mapa más escritura en <code>academico.dat</code>	$\log n + 1$	$O(\log n)$
consultarExpedienteCompleto(carne)	Búsqueda en el mapa más 3 lecturas (identidad, académico, contacto)	$\log n + 3$	$O(\log n)$
<i>Módulo reportes (agregados y mantenimiento)</i>			
calcularPromedioInstitucional()	Recorrido secuencial con contador	n sumas más 1 división	$O(n)$
encontrarMayorPromedio()	Recorrido secuencial con máximo	n comparaciones	$O(n)$
encontrarMayorCreditos()	Recorrido secuencial con máximo	n comparaciones	$O(n)$
generarReporteEstadistico()	Recorrido secuencial con múltiples contadores (por carrera, por nivel, etc.)	$n \times c$, c = cantidad de categorías	$O(n)$

Explicación de cómo se almacenará la información en un archivo secuencial

La información permanente vive en tres archivos binarios de registros de tamaño fijo:

- `identidad.dat`
- `academico.dat`
- `contacto.dat`

Uno por cada struct definido en la sección de diseño del registro.

Como se menciono, tamaño fijo es lo que hace posible calcular la posición física de cualquier registro sin recorrer el archivo:

```
offset = posicion_logica * sizeof(struct)
```

Con ese `offset`, se posiciona el cursor de lectura o escritura directamente sobre el registro deseado, y se ejecuta la acción (leer o escribir) sin tocar el resto del archivo.

La figura 4 ilustra el cálculo: dado un índice i , el `offset` señala exactamente el inicio del registro i -ésimo dentro del archivo, y solo ese registro se trae a RAM con `fread`.

Reforzando la sección anterior, en ningún momento se carga el archivo completo a memoria: solo se traen a RAM los registros necesarios en el momento, siguiendo la posición que entrega el mapa de posiciones

Los tres archivos comparten la misma posición lógica por estudiante, de modo que el registro en la posición i de cada uno pertenece siempre al mismo carné, sin necesidad de guardar una llave adicional dentro de cada struct para relacionarlos entre sí.

La figura 5 muestra esa alineación: la posición i en `identidad.dat`, `academico.dat` y `contacto.dat` corresponde siempre al mismo estudiante, sin necesitar una llave compartida entre los tres archivos.

Propuesta de un archivo indexado que permita acelerar las búsquedas

Recorrer secuencialmente un archivo de cientos de miles de registros para ubicar un solo estudiante cuesta $O(n)$: es aceptable con 25 000 registros, pero se vuelve lento a la escala que el caso de estudio anticipa.

La solución no es cargar todos los registros a memoria como se descartó antes, sino mantener en RAM una estructura mucho más liviana que solo indique dónde está cada registro en disco: un archivo indexado.

Funcionamiento general del mapa de posiciones

El patrón adoptado corresponde a ISAM (*Indexed Sequential Access Method*), una técnica de organización de archivos desarrollada originalmente por IBM en la década de 1960 que combina acceso secuencial y directo mediante un índice sobre un archivo de datos ordenado (GeeksforGeeks, 2025). Silberschatz et al. describen este mismo concepto bajo el nombre *index-sequential file*, como uno de los esquemas de índice más antiguos usados en sistemas de bases de datos, cuyo rendimiento se degrada al crecer el archivo y que por eso fue superado por el índice B+-tree, aunque sin usar la sigla ISAM (Silberschatz et al., 2019).

En este informe nos referimos a esta estructura simplemente como el **mapa de posiciones**, porque describe mejor lo que hace: para cada llave de búsqueda (carné, identificación o apellido) guarda únicamente la posición física del registro en disco, manteniendo aparte un archivo con los datos ordenados según esa misma llave.

Con esa estructura se puede saltar directo al registro deseado en vez de recorrer el archivo completo.

La parte que le da a ISAM su nombre completo (*Indexed Sequential Access Method*) es cómo maneja las inserciones. En vez de reescribir el archivo ordenado cada vez que entra un estudiante nuevo, se ingresa el nuevo valor en una zona aparte sin ordenar, a la que llamamos **lista de recién ingresados**, y se integran a la zona ordenada más adelante, cuando esa acumuló suficientes estudiantes.

Mapa por carné

Este mapa por carné se divide en dos partes:

- Zona ordenada: arreglo ordenado, donde se hace búsqueda binaria en $O(\log n)$.
- Lista de recién ingresados: arreglo sin ordenar para los estudiantes recién registrados en $O(1)$, revisado de principio a fin cuando la zona ordenada no encuentra la llave buscada.

La figura 6 muestra un ejemplo concreto de este flujo: al buscar el carné 1007 sobre la zona ordenada, la búsqueda binaria primero evalúa el índice 2 (carné 1004); como 1007 es mayor, descarta la mitad izquierda y continúa en la mitad derecha, donde el índice 3 (carné 1007) coincide y termina la búsqueda en dos comparaciones. Si ninguna comparación hubiera coincidido, el algoritmo caería en la lista de recién ingresados y la recorrería de principio a fin, como lo indican los carnés 1015, 1003 y 1020 del ejemplo.

Cada entrada del mapa ocupa únicamente el carné, la posición en disco y el estado, no el registro completo. Eso es lo que permite que el mapa quepa en RAM aunque los datos completos no quepan.

Mapas por identificación y por apellido

El caso de estudio exige buscar también por identificación y por apellido, ninguna de las dos es la llave primaria. En vez de recorrer el archivo completo para esas búsquedas, se agregan dos mapas adicionales que apuntan la misma posición física en `identidad.dat`

- Mapa por identificación: funciona igual que el mapa por carné, con zona ordenada y lista de recién ingresados, porque buscar por identificación pasa tan seguido como buscar por carné (en teoría)
- Mapa por apellido: no se actualiza con cada estudiante nuevo como los otros dos. En vez de eso se hace la primera vez que se pide una búsqueda o un reporte por apellido después de que hubo cambios desde la última vez que se armó (también al apagar para guardar el estado).

La figura 7 muestra un ejemplo concreto: la entrada del mapa por carné (1002), la del mapa por identificación (118640022) y la del mapa por apellido (Alvarado) apuntan las tres a la posición 1 de `identidad.dat`, donde vive el mismo registro físico (Ana Alvarado). No existen tres copias de los datos, solo tres formas distintas de localizar la misma posición.

El mapa por apellido funciona así ya que típicamente son para realizar reportes o en general algo que sucede de vez en cuando, podría ejecutarse el sistema y no usarse, por que armar un mapa nuevo que se actualice por cada estudiante no vale la pena

Este diseño donde un dato físico tiene varios mapas apuntando a él, es similar al patrón de una base de datos SQL donde la tabla vive en un lugar y los índices indican donde buscar.

Reorganización y persistencia

Cuando la lista de recién ingresados del mapa por carné o del mapa por identificación llega al 10 % del tamaño (elegido arbitrariamente) de su zona ordenada, se ejecuta una reorganización: como la zona ordenada (n) ya está ordenada, solo se ordenan los k recién ingresados y se mezclan linealmente con ella, en $O(n + k \log k)$, y la lista queda vacía otra vez. Esta reorganización ocurre solo en la estructura del mapa en RAM.

Como esta reorganización ocurre cada $0,1n$ inserciones, el costo amortizado de `registrarPosicionEnMapa` (y por extensión de `registrarEstudiante`) no es $O(1)$: es

$$\frac{O(n + k \log k)}{0,1n} = O(\log n)$$

porque el costo dominante de la reorganización es un ordenamiento, no una copia lineal como en el duplicado clásico de un arreglo dinámico (donde sí se obtiene $O(1)$ amortizado).

En la estructura del estudiante se definió un valor de estado para cada registro, que indica si el estudiante está activo o inactivo. Estos deben borrarse del sistema de archivos ya que son basura; por ello al apagar el sistema:

- Se purgan los registros inactivos de los tres archivos de datos
- Se reconstruyen los tres mapas desde cero a partir del archivo ya compactado

La razón es que la purga cambia la posición física de todos los registros que estaban después de uno eliminado, esto deja inválido cualquier mapa armado antes de purgar, sin importar si ya estaba reorganizado o no.

Los tres mapas completos se guardan en el archivo `estudiantes.idx` al apagar, para que la próxima vez que se encienda el sistema alcance con cargar los mapas ya armados en $O(n)$, en vez de reconstruirlos desde los archivos de datos.

Diagramas

- **Layout del registro** (figura 1, sección): los campos de los tres structs
(IdentidadEstudiante, AcademicoEstudiante, ContactoEstudiante)
- **Mapa de posiciones en RAM** (figura 2, sección): la zona ordenada y la lista de recién ingresados del mapa por carné.
- **Relación RAM-disco** (figura 3, sección): cómo cada entrada del mapa en RAM apunta, mediante `offset`, a una posición en los tres archivos en disco.
- **Cálculo del `offset`** (figura 4, sección): cómo se ubica un registro dentro de un archivo de tamaño fijo a partir de su posición lógica.
- **Alineación por posición lógica** (figura 5, sección): los tres archivos binarios comparten posición lógica por estudiante, sin necesitar una llave adicional entre ellos.
- **Estructura interna de los mapas** (figura 6, sección): el flujo de búsqueda binaria en la zona ordenada y lineal en la lista de recién ingresados.
- **Un registro, tres mapas** (figura 7, sección): los mapas por carné, identificación y apellido apuntando todos al mismo registro físico.

Mapa conceptual

El siguiente mapa conceptual relaciona los ocho conceptos centrales de esta investigación, siguiendo las mismas decisiones de diseño explicadas en el resto del informe.

Un **Dato** sin clasificar no dice mucho por sí solo: primero se clasifica según su **Tipo de Dato Abstracto** (entero, cadena, fecha, enumerado), y ese tipo es lo que define cómo se declara cada campo del **Registro**. Varios registros consecutivos forman un **Arreglo**, que en este sistema vive tanto en disco (los tres archivos de tamaño fijo) como en RAM (el mapa de posiciones).

Sobre ese arreglo operan las **Funciones** documentadas en las secciones anteriores, y cada una de ellas se analiza según su **Costo computacional** en notación Big O, tal como exige el caso de estudio.

Los registros no viven solo en memoria: se guardan de forma permanente en un **Archivo secuencial**, accesible por posición mediante funciones como `fseek` y `fread`. Para no tener que recorrerlo completo en cada búsqueda, ese archivo se acelera agregando un **Archivo indexado**, que a su vez es justamente el que mantiene en RAM el arreglo (mapa de posiciones) mencionado antes, cerrando el ciclo entre los ocho conceptos.

Conclusiones

El caso de la Universidad XUNA muestra que los problemas administrativos descritos al inicio (expedientes duplicados, información inconsistente, búsquedas lentas) no se resuelven simplemente centralizando los datos, sino decidiendo con cuidado cómo se representan y cómo se accede a ellos. Esa fue la línea que guió todo el diseño propuesto en este informe.

Separar el registro en tres structs, en vez de un bloque único, salió de mirar qué tan seguido cambia cada dato y quién lo usa junto con qué, no de una preferencia de diseño.

Lo mismo con el mapa de posiciones: no tiene sentido cargar 25 000 expedientes completos en RAM si alcanza con guardar el carné, la posición en disco y el estado de cada uno.

El patrón ISAM terminó siendo lo que conecta arreglo, archivo secuencial y archivo indexado entre sí: la zona ordenada resuelve la búsqueda en $O(\log n)$, la lista de recién ingresados evita reordenar el archivo en cada matrícula nueva, y el offset calculado a partir del tamaño fijo del registro es lo que permite llegar directo a un registro en disco sin recorrer los demás.

En conjunto, estas decisiones muestran que el diseño de estructuras de datos no depende únicamente de que una solución funcione, sino de que su costo computacional se mantenga razonable a medida que la cantidad de dato crece. Ese es el criterio que se aplicó en cada función documentada a lo largo del informe, y el que deberá mantenerse durante la implementación del sistema en el resto del semestre.

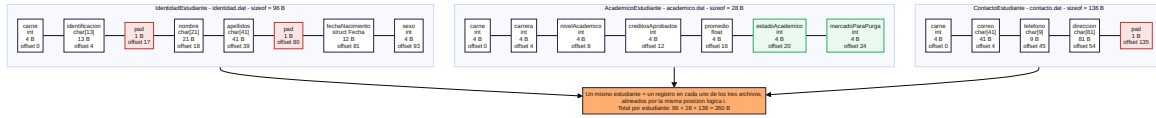


Figura 1

Layout de campos de los tres structs del registro Estudiante, con offsets internos (incluye el padding de alineación)

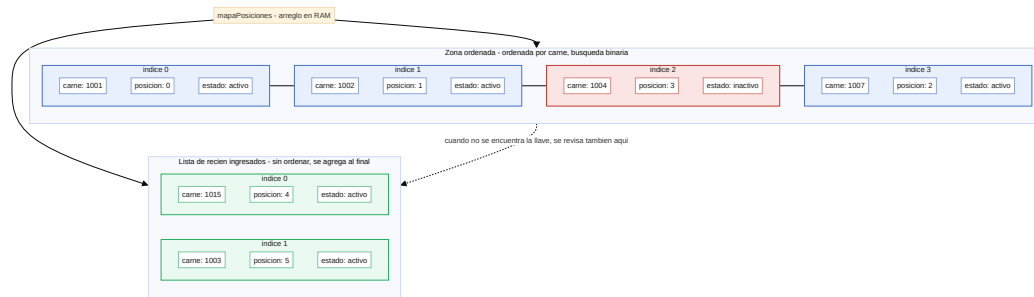


Figura 2

El mapa de posiciones como arreglo en RAM

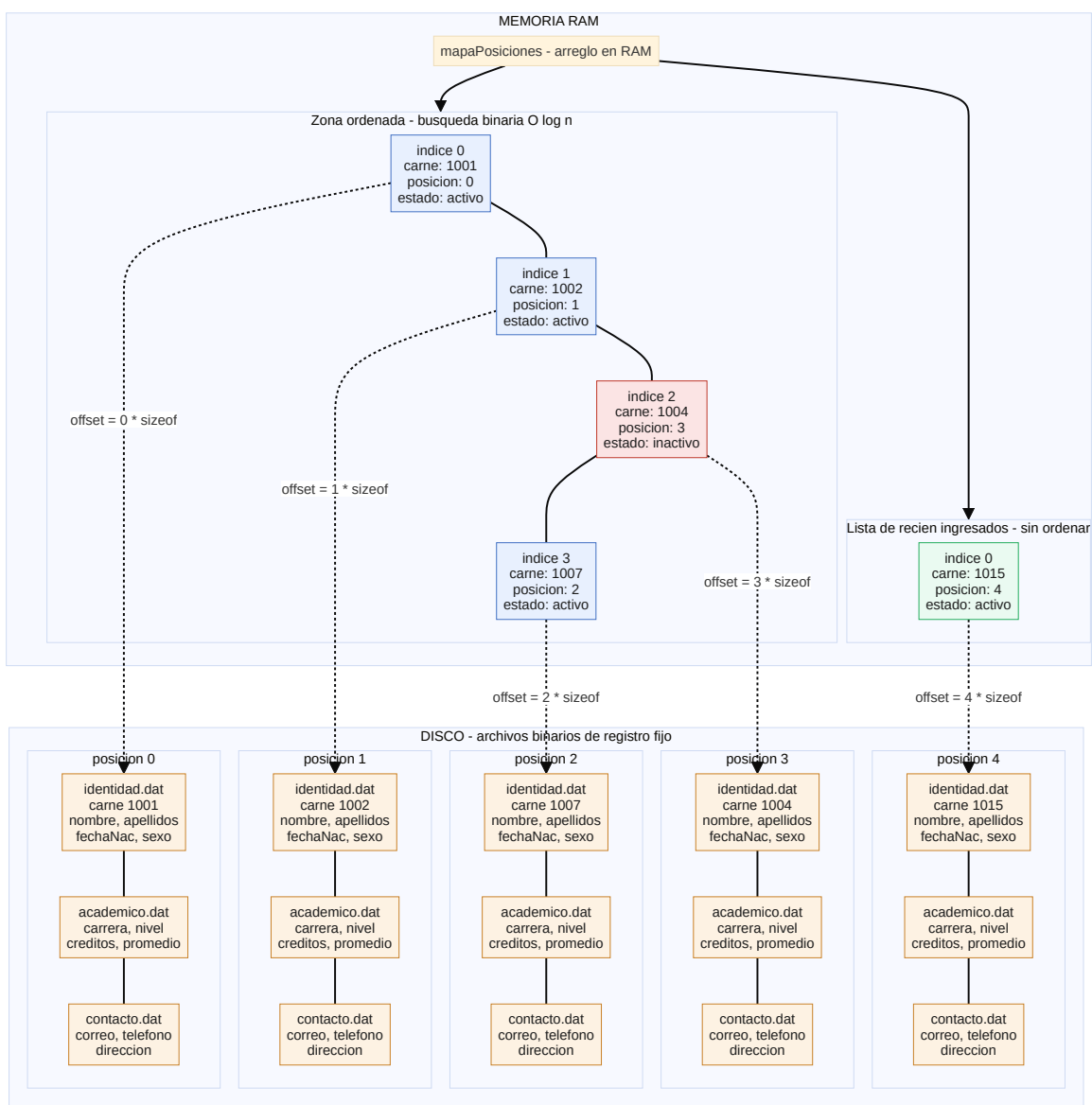


Figura 3

Relación entre el mapa de posiciones en RAM y los archivos en disco

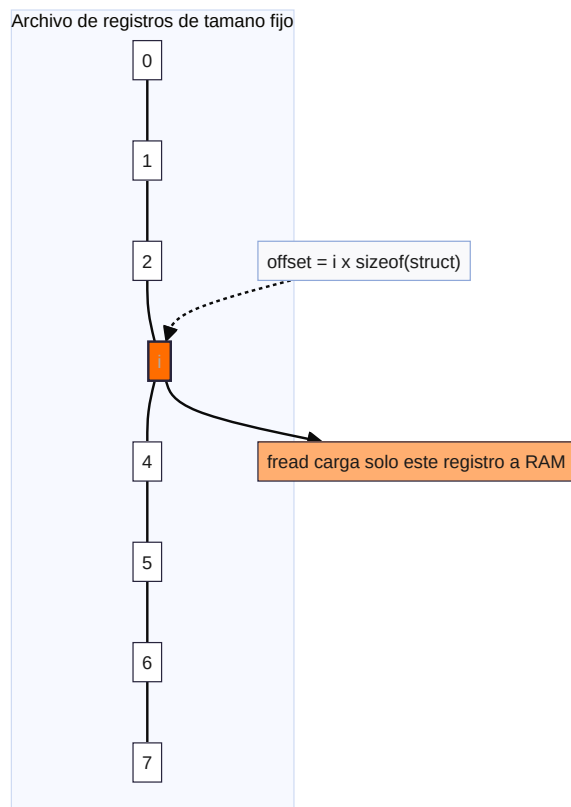


Figura 4

Cálculo del `offset` sobre un archivo de registros de tamaño fijo

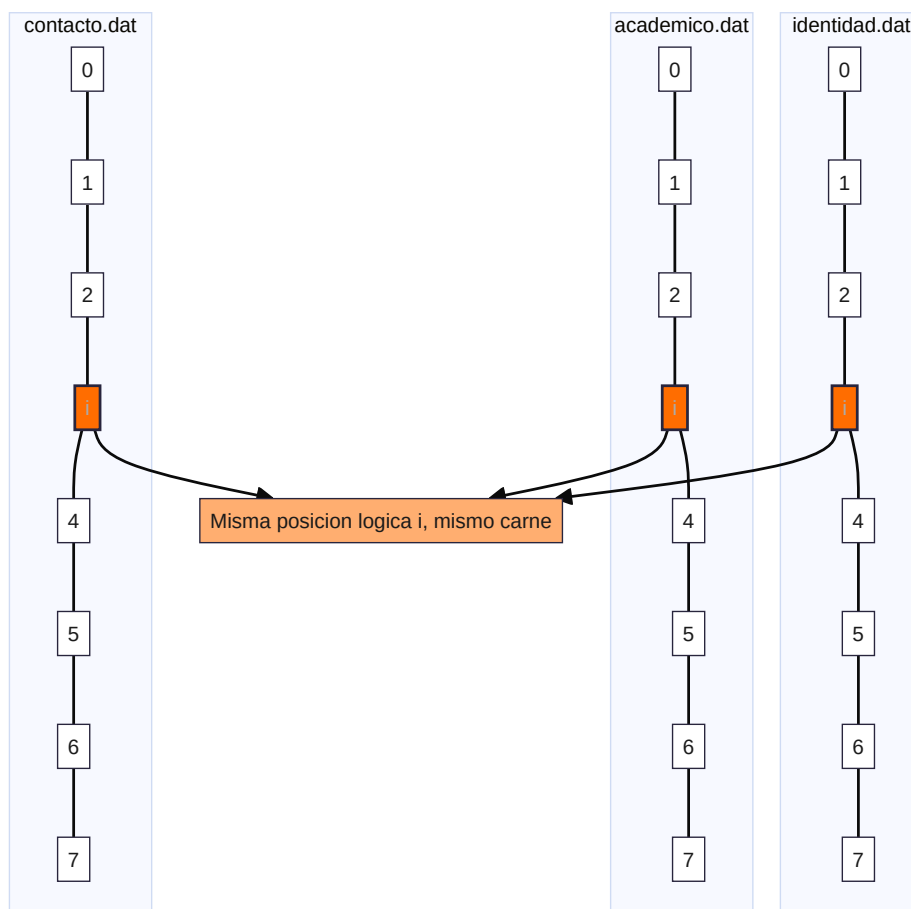


Figura 5

Los tres archivos alineados por la misma posición lógica `i`

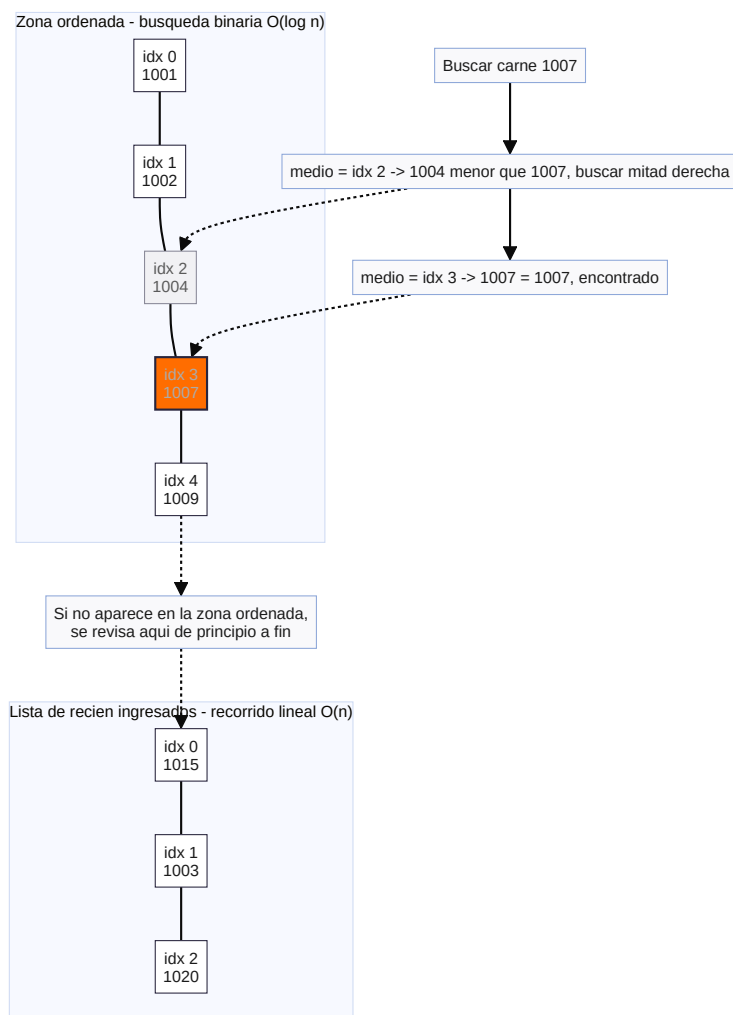


Figura 6

Estructura interna del mapa por carné o identificación: zona ordenada y lista de recién ingresados

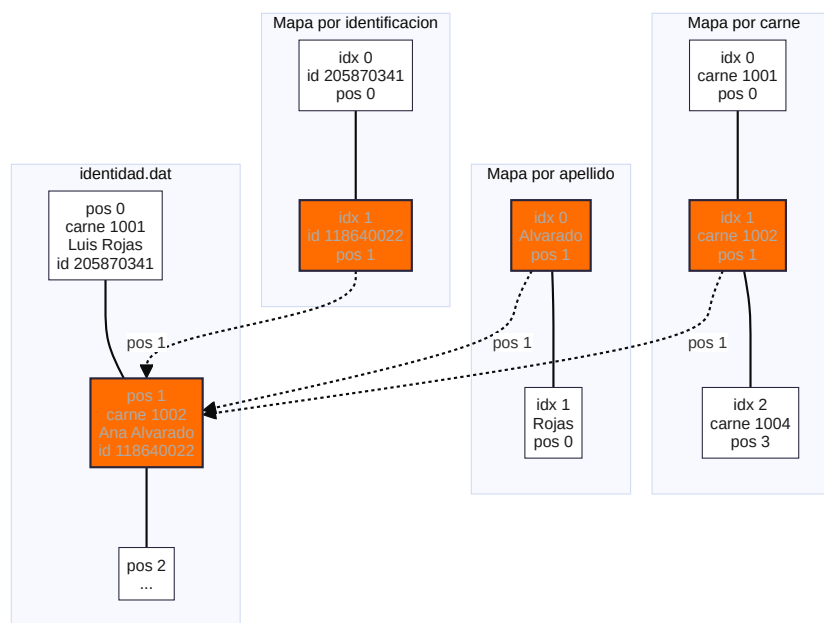


Figura 7

Un mismo registro físico referenciado por los tres mapas: carné, identificación y apellido

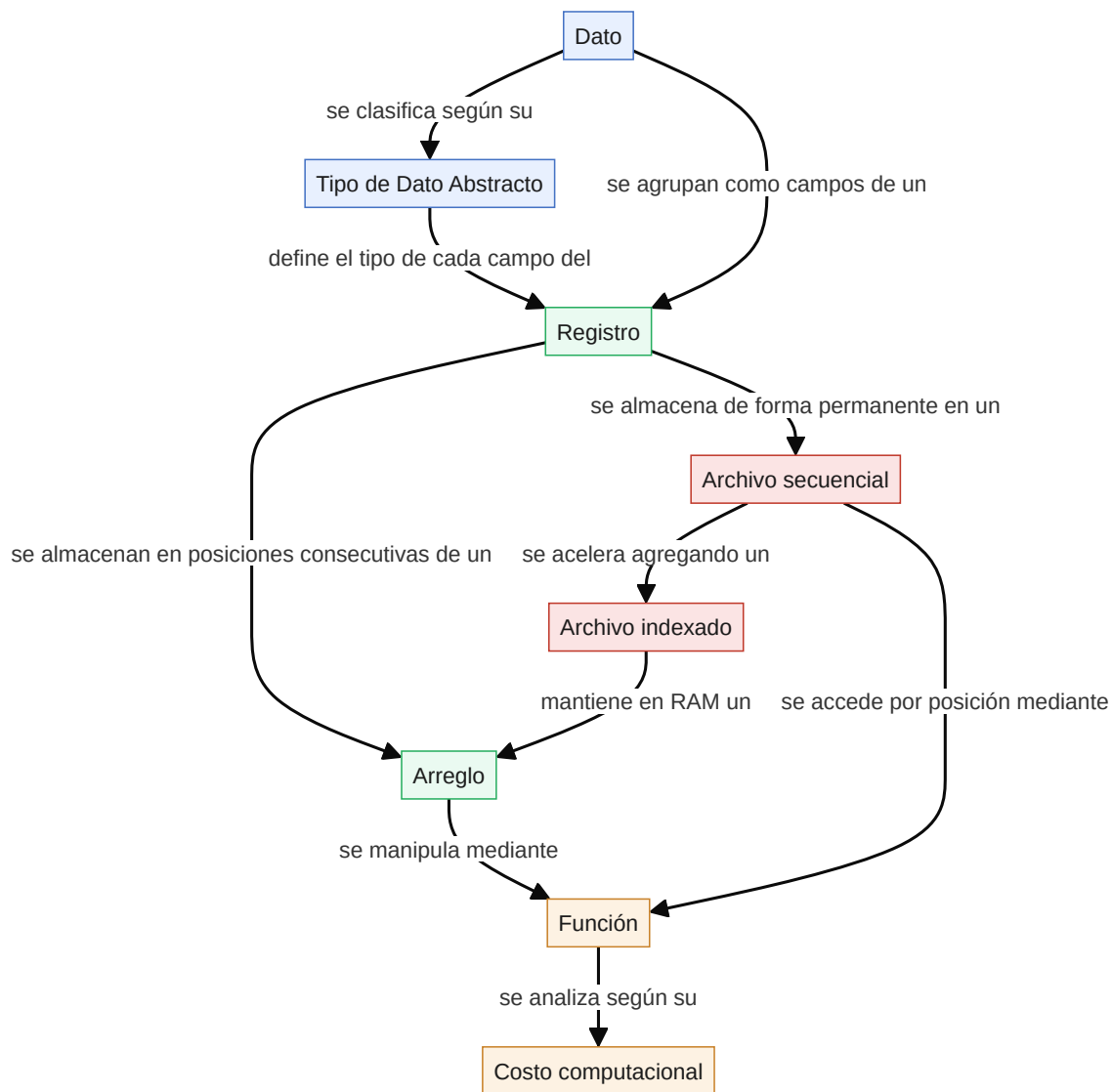


Figura 8

Mapa conceptual de la investigación

Referencias

- GeeksforGeeks. (2025). ISAM in Database [Explicación de ISAM y su relación con árboles B y B+]. Consultado el 9 de agosto de 2026, desde <https://www.geeksforgeeks.org/dbms/isam-in-database/>
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7.^a ed.) [Véase el Capítulo 14, “Indexing”: sección sobre (“index-sequential file”)]. McGraw-Hill Education. Consultado el 9 de agosto de 2026, desde <https://www.mpgcamb.com/wp-content/uploads/2024/12/Abraham-Silberschatz-Henry-F.-Korth-S.-Sudarshan-Database-System-Concepts-McGraw-Hill-Education-2019.pdf>